

Acid Zero - Hardware & Porting Reference

Raspberry Pi 3B+ pentest handheld - for porting C/C++/Rust/Python/Flipper code

Raspberry Pi 3 Model B Plus Rev 1.4 (rev a020d4)
Debian GNU/Linux 13 (trixie) kernel 6.12.62+rpt-rpi-v8 aarch64
generated 2026-06-27 21:06

1. SYSTEM OVERVIEW (live)

Board : Raspberry Pi 3 Model B Plus Rev 1.4 (rev a020d4)
OS : Debian GNU/Linux 13 (trixie) kernel 6.12.62+rpt-rpi-v8 (aarch64)
RAM : 955 MB
Active dtoverlays: dtoverlay=disable-wifi dtoverlay=dwc2 dtoverlay=dwc2,dr_mode=host
dtoverlay=piscreen,speed=16000000,rotate=270 dtoverlay=spi0-2cs dtoverlay=vc4-kms-v3d
I2C buses: /dev/i2c-1 /dev/i2c-2 SPI char devs: none (SPI0 consumed by TFT overlay)

2. DISPLAY - ILI9486 TFT

Panel: ILI9486 480x320 16bpp RGB565. Driver: fbttf (piscreen overlay). Node: /dev/fb1.
Bus: SPI0 @ 16 MHz, rotate 270. Overlay: dtoverlay=piscreen,speed=16000000,rotate=270

RGB565 pixel format (what every native app must write)

```
v = ((R & 0xF8) << 8) | ((G & 0xFC) << 3) | (B >> 3) # 16-bit  
byte order = little-endian, 2 bytes/px. frame = 480*320*2 = 307200 bytes  
pixel (x,y) byte offset = (y*480 + x) * 2
```

To draw: build a 480x320 RGB565 buffer and write the whole frame to /dev/fb1 (or mmap it).

SPI wiring (piscreen standard, SPI0)

MOSI GPIO10 (pin19) SCLK GPIO11 (pin23) LCD-CS CE0 GPIO8 (pin24)
TOUCH-CS CE1 GPIO7 (pin26) DC GPIO24 RESET GPIO25 backlight GPIO18

ILI9486 command set (bare-metal SPI port)

```
0x01 SWRESET 0x11 SLP_OUT 0x29 DISP_ON 0x36 MADCTL(rotation) 0x3A COLMOD(0x55=16bpp)  
0x2A CASET(col) 0x2B PASET(page) 0x2C RAMWR(pixel data follows)
```

3. TOUCH - ADS7846 / XPT2046

event0: ADS7846 Touchscreen on SPI0 CE1 (GPIO7) + IRQ GPIO. Linux input event device, 12-bit X/Y.

Acid Zero uses 4-point AFFINE calibration (least-squares), residual gate <= 45 px.

Raw read (bare-metal SPI)

TX 0x90 -> read 2 bytes -> Y (>>3 = 12-bit). TX 0xD0 -> read 2 bytes -> X (>>3).

Linux input_event struct (read from /dev/input/eventN)

```
{ struct timeval time; __u16 type; __u16 code; __s32 value; } (16B on 32-bit / 24B 64-bit)  
EV_SYN=0 EV_KEY=1 EV_ABS=3 | ABS_X=0x00 ABS_Y=0x01 | BTN_TOUCH=0x14a
```

4. NATIVE PLUGIN CONTRACT (add ANY app, incl compiled C/C++/Rust)

Drop a folder: /usr/local/lib/acid-apps/<yourapp>/ containing app.json + an executable.

app.json manifest

```
{ "type": "native", "name": "My App", "icon": "usb",  
  "color": [30, 200, 121], "exec": "mybin" } # exec = binary or script in the folder
```

Runtime contract

- 1) Your program OWNS /dev/fb1 + the touch event device while it runs.
- 2) Render RGB565 frames (see section 2); read taps from the ADS7846 evdev node.
- 3) EXIT(0) on your exit gesture. The launcher is BLOCKED on you and redraws home on return.

Find devices by NAME, not number (fb#/event# reshuffle each boot):

```
fb -> match 'ili9486' / 'fb_ili9486' in /sys/class/graphics/fb*/name
```

```
touch -> match 'ADS7846' in /sys/class/input/event*/device/name
```

Language-agnostic: anything that can open /dev/fb1 + an evdev node works (C/C++/Rust/Go/Py).

Reference template: /usr/local/lib/acid-apps/hello-native/ (standalone Python).

Python in-process plugins (lighter alternative)

/usr/local/lib/acid-apps/*.py with META={name,icon,color} + draw(d,ctx)

optional: on_enter(ctx), handle_touch(tx,ty,ctx). Shares the launcher Ctx (fonts/colors/helpers).

5. WiFi RADIOS

Bind adapters by USB VID:PID, NEVER by wlanN (interface numbers reshuffle every boot).

VID:PID chipset module bands role
onboard BCM43455 built-in 2.4/5 limited monitor
0e8d:7612 MT7612U AWUS036ACM 2.4/5 monitor+inject
0bda:8812 RTL8812AU AWUS036ACH 2.4/5 AP+inject
2357:0120 RTL8821AU Archer T2U+ 2.4/5 client/SSH
0bda:8179 RTL8188EUS generic 2.4 only client

live interface -> driver map

wlan0 brcmfmac
wlan0mon mt76x2u
wlan1 rtl8821au
wlan2 rtl8812au
wlan3 mt76x2u
wlan4 rtl8xxxu

mt76x2u exposes a base iface + a monitor iface (wlan0mon). brcmfmac = onboard BCM43455.

6. BLUETOOTH / BLE

Controller: hci0, Broadcom over UART, BD addr B8:27:EB:59:00:11.

Acid Zero BLE spam uses a RAW HCI USER-channel socket (HCI_CHANNEL_USER=1); bluetoothd is stopped

for exclusive control. ~32 adv frames/sec. own_addr_type MUST be 01 (random) or iOS dedups.

Key HCI LE commands (OGF 0x08)

OCF 0x0005 LE Set Random Address 0x0006 LE Set Adv Params 0x0008 LE Set Adv Data
OCF 0x000A LE Set Adv Enable ioctl HCIDEVDOWN = 0x400448ca

BLE advertising payloads implemented (Flipper BLE-spam port reference)

Apple Continuity : mfg Company 0x004C; ProximityPair type 0x07 + 2B model; NearbyAction 0x0F flags 0xC0
Google Fast Pair : 02 01 06 | 03 03 2C FE | 06 16 2C FE <3B model> (Google validates model IDs)
Samsung : mfg Company 0x0075 Microsoft Swift Pair: mfg Company 0x0006

BLE scan (read-only recon)

btmon decodes each adv report (Company-ID->vendor, OUI, name, appearance, Apple model, Fast-Pair model).

drive discovery: sleep N | timeout M btmgmt find -l (keep stdin open; btmgmt stops on EOF)

7. SUB-GHz - CC1101 (planned: porting guide)

CC1101 = SPI sub-GHz transceiver: 300-348 / 387-464 / 779-928 MHz. Same family Flipper uses, so .sub OOK/ASK captures are conceptually portable.

WIRING PROBLEM on this build

SPI0 is fully consumed by the TFT + touch (piscreeen). For CC1101 use one of:

a) SPI1 - enable dtoverlay=spi1-1cs : MISO GPIO19(p35) MOSI GPIO20(p38) SCLK GPIO21(p40) CE GPIO18/16/17

(watch GPIO18 vs backlight clash)

b) bit-bang SPI on spare GPIOs c) time-share SPI0 2nd CS (risky during display refresh)

CC1101 register/strobe basics

FREQ2/1/0 carrier MDMCFG2 (ASK/OOK) PKTCTRL0/1 DEVIATN AGCCTRL FSCAL TEST
strokes: SRES 0x30 SRX 0x34 STX 0x35 SIDLE 0x36 SFRX 0x3A SFTX 0x3B FIFO 0x3F(burst)
freq: f = FREQ * 26MHz / 2^16 433.92 MHz -> FREQ ~= 0x10B071

Stacks: SmartRC-CC1101 (Arduino port), pi433, python cc1101 over spidev. OOK replay via GDO0 timing.

Flipper .sub porting: parse RAW_Data timings -> toggle GDO0 TX (timed) or CC1101 async TX.

8. NFC / RFID - PN532 (planned: porting guide)

PN532 = 13.56 MHz NFC: ISO14443A/B, MIFARE Classic/Ultralight, FeliCa, NDEF, card emulation.

Selectable bus by DIP: I2C / SPI / HSU(UART).

RECOMMENDED here: I2C (I2C-1 is FREE; SPI0 is busy with the TFT)

SDA GPIO2 (pin3) SCL GPIO3 (pin5) VCC 3V3 GND addr 0x24 (IRQ optional GPIO)

Software: libnfc (device = pn532_i2c:/dev/i2c-1), Adafruit-PN532 (python I2C), nfcpy (UART).

MIFARE Classic keys: mfoc / mfcul + libnfc.

NOTE: 125 kHz RFID (EM4100/HID) needs a DIFFERENT front-end (RDM6300 UART / EM4095) - PN532 is

13.56 MHz only. (Flipper covers both bands via separate hardware front-ends.)

Flipper NFC porting: dump = UID + sector keys + blocks; replay via PN532 emulation or a magic card.

9. IR (planned)

TX: 38 kHz carrier via dtoverlay=gpio-ir-tx (a TX GPIO). RX: dtoverlay=gpio-ir.

Tools: LIRC or ir-ctl. Flipper .ir (protocol/address/command) -> LIRC config or raw timing.

10. GPIO / BUS AVAILABILITY MAP (this build)

SPI0 : BUSY - ILI9486 (CE0) + ADS7846 touch (CE1)

SPI1 : FREE - candidate for CC1101 (enable dtoverlay=spi1-1cs)

I2C-1 : FREE - candidate for PN532 (GPIO2/3)

I2C-2 : present

UART0 : GPIO14/15 - console/BT, handle with care

USB : 4x WiFi dongles via onboard hub

free GPIO : 5 6 12 13 16 17 19 20 21 22 23 26 27 (verify vs active overlays before use)

11. PORTING FLIPPER CODE -> Acid Zero (cheat-sheet)

UI : Flipper Canvas 128x64 mono -> fb1 480x320 RGB565 (map canvas_draw_* to PIL/fb)

Buttons : Flipper D-pad/OK -> ADS7846 taps (evdev)

Sub-GHz : Flipper CC1101 -> Pi CC1101 on SPI1 (same chip: port reg init + .sub)

NFC : Flipper ST25R3916 -> PN532 (diff chip: port logic via libnfc, not regs)

BLE spam: apple_ble_spam.c payloads -> already done via raw HCI (section 6)

IR : .ir files -> LIRC / ir-ctl

Apps : compile to ARM64 (aarch64) -> drop as native plugin (section 4)